

Hibernate Configuration Using Java Without XML file

In the provided code, we configure Hibernate directly in Java instead of using the traditional hibernate.cfg.xml file. Here's a breakdown of the important points with explanations.

Code:

```
Configuration config = null;  
SessionFactory sessionFactory = null;  
Session session = null;  
Transaction transaction = null;  
boolean flag = false;  
  
config = new Configuration();  
  
// Setting Hibernate properties directly using setProperty method  
config.setProperty("hibernate.connection.driver_class",  
"com.mysql.cj.jdbc.Driver");  
config.setProperty("hibernate.connection.url",  
"jdbc:mysql://localhost:3306/telusko_db");  
config.setProperty("hibernate.connection.password", "root123");  
config.setProperty("hibernate.connection.username", "root");  
config.setProperty("hibernate.dialect", "org.hibernate.dialect.MySQLDialect");  
config.setProperty("hibernate.hbm2ddl.auto", "update");  
config.setProperty("hibernate.show_sql", "true");  
config.setProperty("hibernate.format_sql", "true");  
  
// Adding annotated class  
config.addAnnotatedClass(Student.class);  
  
// Building session factory  
sessionFactory = config.buildSessionFactory();
```

```
// Opening a session  
session = sessionFactory.openSession();
```

Explanation of Important Points:

1. **Configuration config = null;**
 - **Purpose:** A Configuration object is used to specify all the configuration details that would normally be in the hibernate.cfg.xml file.
 - **Importance:** Here, we are not using XML files, so the configuration is directly defined in the Java code.

Advantages of Configuring Hibernate Without XML:

1. **Code-Centric Approach:** You avoid XML-based configuration, keeping everything within your code. This can be more intuitive for developers who prefer code over external configuration.
2. **Flexibility:** You can dynamically set properties or adjust them programmatically based on the environment (e.g., different database URLs for development and production).
3. **Simplified Management:** For smaller applications, it might simplify the setup since all the configuration is in one place, reducing the need for external files.

Key Notes:

- **Avoid Hardcoding:** It's good practice to avoid hardcoding sensitive information like database passwords. Consider using environment variables or a configuration management system.